FlashAttention em CUDA: Implementação e Avaliação de um Kernel de Atenção com Tiling e Softmax Online em GPU de Entrada

Proposta de Projeto Final — Programação e Arquitetura de Computadores Paralelos (DCC030/049)
Iago Nathan — Prof. George Luiz Medeiros Teodoro — 30/09/2026

1 Problema a ser implementado

O mecanismo de atenção (*scaled dot-product attention*) é o componente central da arquitetura Transformer, hoje a base de praticamente todos os grandes modelos de linguagem e de visão computacional. Sua formulação padrão calcula $\text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$, o que exige a materialização explícita de uma matriz de scores de tamanho $N \times N$ (N = comprimento da sequência) na memória global da GPU (HBM). Esse passo faz com que a implementação ingênua seja limitada por largura de banda de memória, e não por capacidade de computação — o número de acessos à HBM cresce quadraticamente com N, mesmo que o número de operações de ponto flutuante necessárias não precisasse crescer dessa forma.

Este projeto propõe reimplementar, em CUDA e a partir do zero, a ideia algorítmica central do FlashAttention (Dao et al., NeurIPS 2022): um kernel fundido (*fused*) que particiona as matrizes Q, K e V em blocos (*tiling*) que cabem na memória compartilhada (*shared memory*) do Streaming Multiprocessor, e que mantém uma normalização de softmax incremental ("online softmax") à medida que os blocos de K e V são processados, evitando por completo a materialização da matriz $N \times N$. O resultado é matematicamente idêntico à atenção padrão (não é uma aproximação), porém com uma pegada de memória muito menor.

O escopo do projeto será delimitado da seguinte forma: (i) apenas o *forward pass* (inferência) da atenção multi-cabeça não causal, em precisão FP32; (ii) implementação como uma extensão CUDA de PyTorch (kernel `.cu` exposto via `torch.utils.cpp_extension`), permitindo validar correção e medir desempenho a partir de Python; (iii) avaliação em três versões — uma implementação ingênua que materializa a matriz de scores, o kernel com tiling e softmax online desenvolvido neste projeto, e a implementação nativa otimizada do PyTorch (`scaled_dot_product_attention`) como referência superior; (iv) correção validada por tolerância numérica contra uma implementação de referência, já que somas de ponto flutuante em ordens diferentes não produzem bits idênticos. Os experimentos serão conduzidos em uma GPU NVIDIA T4 (arquitetura Turing) disponível gratuitamente via Google Colab.

2 Relevância

O custo quadrático da atenção é um dos principais fatores limitantes para o uso de modelos de linguagem com contextos longos, e a técnica aqui estudada — em versões mais avançadas (FlashAttention-2 e FlashAttention-3) — é hoje parte da infraestrutura real de inferência e treinamento de praticamente todos os grandes modelos de linguagem em produção. O projeto também se conecta diretamente a vários tópicos já vistos na disciplina: arquitetura de GPUs e programação CUDA, processamento data-parallel, e as otimizações de desempenho relacionadas à comunicação e à localidade de dados na hierarquia de memória. Trata-se, portanto, de uma aplicação não trivial cuja relevância é tanto acadêmica (ilustra de forma muito concreta o trade-off entre computação e movimentação de dados em GPUs) quanto prática (a técnica é usada amplamente na indústria).

3 Dificuldades do ponto de vista da paralelização

(a) Natureza limitada por memória: na versão ingênua, o gargalo não é o número de operações de ponto flutuante, e sim o volume de leituras e escritas na memória global para armazenar a matriz intermediária $N \times N$. Reduzir esse tráfego exige reestruturar o algoritmo em torno da hierarquia de memória da GPU (registradores, memória compartilhada, memória global), e não apenas paralelizar o código original.

(b) Softmax online como desafio de correção, não só de desempenho: a normalização softmax exige, em princípio, conhecer a soma de toda uma linha antes de finalizar qualquer saída. Processar blocos de K/V de forma incremental exige manter um máximo corrente e uma soma corrente de exponenciais, e reescalar as saídas parciais a cada novo bloco (técnica log-sum-exp) — um padrão de redução incomum, cuja implementação incorreta produz resultados numericamente instáveis ou errados sem que isso seja óbvio a partir apenas do tempo de execução.

(c) Ajuste fino de ocupação da GPU: o tamanho dos blocos (*tile size*) determina um compromisso entre uso de memória compartilhada, pressão de registradores e número de warps ativos por Streaming Multiprocessor. Esse ajuste é empírico e requer experimentação cuidadosa.

(d) Ambiente de execução: Os experimentos serão executados em plataformas gratuitas na nuvem (Google Colab / Kaggle Notebooks). Nessas plataformas, perfiladores de contadores de hardware (Nsight Compute, nvprof) tipicamente não funcionam por falta de acesso privilegiado ao driver, de modo que a avaliação experimental será baseada em medições de tempo de parede via eventos CUDA e em uso de memória de pico, em função do comprimento da sequência (por exemplo, de 128 a 4096/8192 tokens), comparando as três versões descritas na Seção 1.

Referências

- T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022.
- T. Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR, 2023.
- J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, T. Dao. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*. NeurIPS, 2024.